

Exploratory Data Analysis & Data Cleaning — The Complete Study Guide

Pandas & NumPy | From Beginner to Interview-Ready

"Garbage in, garbage out." Every model, dashboard, and business decision is only as good as the data underneath it. This guide teaches you how to make that data trustworthy — one clean dataset at a time.

Datasets used throughout this guide (deliberately **not** Titanic): **Housing Prices, Customer Information, Sales Data, Medical Records, Employee Records, Restaurant Reviews, Banking Transactions, Product Inventory, and E-commerce Orders**. Each chapter picks whichever dataset best illustrates the concept, so by the end you've cleaned nine different flavors of real-world mess, not one dataset nine times.

How to Use This Guide

- Read top to bottom the first time — chapters are ordered the way you'd actually clean a dataset.
- Come back to individual chapters before interviews using the **Pro Tip** / **Warning** / **Common Mistake** boxes and the Chapter 15 cheat sheet.
- Chapter 20 is a 10-15 minute lightning revision if you're short on time.

Legend used throughout:

-  **Pro Tip** — a habit that separates juniors from seniors
-  **Warning** — something that silently breaks your analysis
-  **Common Mistake** — something almost everyone does at least once
-  **Interview Tip** — what interviewers actually listen for

Table of Contents

1. Introduction to Data Cleaning
 2. Loading & Exploring Data
 3. Understanding Your Dataset
 4. Handling Missing Values (Most Detailed Chapter)
 5. Removing Duplicate Data
 6. Fixing Data Types
 7. Cleaning Text Data
 8. Handling Outliers
 9. Cleaning Categorical Data
 10. Column Operations
 11. Filtering Data
 12. Cleaning Numeric Values
 13. Cleaning Dates
 14. Combining Cleaning Techniques (Full Walkthrough)
 15. Pandas Data Cleaning Cheat Sheet
 16. Common Beginner Mistakes
 17. Interview Questions (45 total, with answers)
 18. Complete Data Cleaning Roadmap
 19. Practice Datasets (Solve Independently)
 20. Final Revision Sheet
-

Chapter 1 — Introduction to Data Cleaning

What is Data Cleaning?

Data Cleaning is the process of fixing or removing incorrect, incomplete, duplicate, inconsistent, or irrelevant data from a dataset. It's the "washing your vegetables before cooking" step of data science — unglamorous, but skip it and everything downstream tastes wrong.

Formally: Data Cleaning transforms **raw data** (messy, as collected from the real world) into **analysis-ready data** (consistent, correctly typed, and trustworthy).

Why Data Cleaning Matters

Studies consistently show data scientists spend **60–80% of their time** cleaning data, not modeling it. Here's why it's worth every minute:

- **Garbage in, garbage out** — even the best machine learning model can't fix biased or broken inputs.
- **Wrong business decisions** — a retailer analyzing "average order value" with currency symbols still in the price column will silently get `NaN` or a crashed script, and might "fix" it by dropping real orders.
- **Legal & compliance risk** — in banking and healthcare, dirty data (wrong patient IDs, mismatched account numbers) can cause real financial or safety harm.

Real-World Examples of Dirty Data

Dataset	What "dirty" looks like
Housing Prices	<code>price</code> column has values like <code>"\$250,000"</code> , <code>250000</code> , and <code>"250K"</code> mixed together
Customer Information	Phone numbers stored as <code>"9876543210"</code> , <code>"98765-43210"</code> , and <code>987654321</code> (one digit missing)
Sales Data	Dates stored as <code>"12/01/2024"</code> in some rows and <code>"2024-01-12"</code> in others — is that January 12th or December 1st?
Medical Records	Patient age listed as <code>-5</code> or <code>250</code>
Employee Records	Department names entered as <code>"Sales"</code> , <code>"sales"</code> , <code>"SALES "</code> , and <code>"Sale"</code>
Restaurant Reviews	Rating column mixing <code>"4 stars"</code> , <code>4.0</code> , and <code>"Excellent"</code>
Banking Transactions	Duplicate transaction rows from a retried API call
Product Inventory	Stock quantity listed as <code>"N/A"</code> instead of <code>0</code> or blank
E-commerce Orders	Customer email <code>"john@gmail..com"</code> — clearly a typo, not a valid contact

Consequences of Poor Data Quality

- Incorrect averages, totals, and trends in reports
- Machine learning models that learn the wrong patterns (or crash on bad input)
- Broken joins/merges between tables (mismatched IDs or inconsistent keys)
- Wasted engineering hours debugging "why is this number wrong" days after launch

- Loss of stakeholder trust — once someone finds one wrong number, they stop trusting the whole report

Data Cleaning vs. Data Preprocessing

These terms are often confused. Here's the clean (pun intended) distinction:

	Data Cleaning	Data Preprocessing
Goal	Make data <i>correct and consistent</i>	Make data <i>model-ready</i>
Includes	Fixing missing values, duplicates, wrong types, typos, outliers	Cleaning plus scaling, encoding, feature engineering, train/test splitting
Scope	Narrower — about correctness	Broader — about correctness and usability for ML
Analogy	Washing and chopping the vegetables	Washing, chopping, and turning them into a specific recipe's ingredients

💡 **Pro Tip:** In interviews, saying "Data Cleaning is a subset of Data Preprocessing" is the single sentence that shows you understand the hierarchy.

Where Data Cleaning Fits in the ML/Analytics Pipeline

```

Business Problem
  ↓
Data Collection (databases, APIs, forms, sensors, files)
  ↓
Data Cleaning ← you are here, and you'll return here often
  ↓
Exploratory Data Analysis (EDA)
  ↓
Feature Engineering
  ↓
Model Building / Dashboard Building
  ↓
Evaluation & Deployment
  
```

⚠️ **Warning:** Data Cleaning is not a one-time step done before EDA — it's a *loop*. EDA often reveals new problems (an outlier you didn't notice, a category typo hiding in a rare value) that send you back to cleaning.

The Typical Workflow Data Scientists Follow

1. Load the dataset and look at its shape and structure
2. Inspect data types, missing values, and duplicates
3. Fix structural issues (wrong types, inconsistent categories, messy text)
4. Handle missing values and duplicates
5. Detect and treat outliers
6. Validate the final dataset against business logic (e.g., "age can't be negative")
7. Save a clean version — **never overwrite your raw data file**

This exact sequence is the backbone of the rest of this guide, and gets its own detailed roadmap in **Chapter 18**.

Chapter 2 — Loading & Exploring Data

Every cleaning project starts the same way: get the data into Pandas, then look at it before touching anything.

Loading Functions

`pd.read_csv()`

What it is: Reads a comma-separated (or other delimiter) text file into a DataFrame. **Why it matters:** CSV is the most common exchange format for tabular data — exports from databases, Excel, and APIs all commonly land as CSV. **Syntax:**

```
python

df = pd.read_csv(filepath_or_buffer, sep=',', header=0, names=None,
                  dtype=None, na_values=None, parse_dates=None, encoding='utf-8')
```

Key parameters:

- `sep` — delimiter (use `'\t'` for tab-separated, `'+'` for European CSVs)
- `dtype` — force column types on load (avoids surprises later)
- `na_values` — tell Pandas which strings mean "missing" (e.g. `["NA", "N/A", "--", "unknown"]`)
- `parse_dates` — parse specific columns as dates immediately

Example — Sales Data:

```
python

sales = pd.read_csv("sales_data.csv", na_values=["N/A", "--"], parse_dates=["order_
```

⚠ **Warning:** By default, Pandas only recognizes a limited list of missing-value strings (like empty string, `NaN`, `NULL`). A dataset using `"unknown"` or `"-"` for missing values will load as valid text unless you tell Pandas about it with `na_values`.

`pd.read_excel()`

What it is: Reads an Excel file (`.xlsx` / `.xls`), one sheet at a time by default. **Syntax:**

```
python

df = pd.read_excel("employee_records.xlsx", sheet_name="2024_Headcount", engine="op
```

💡 **Pro Tip:** `sheet_name=None` returns a dictionary of **all** sheets — useful when a workbook has one sheet per month/region.

`pd.read_json()`

What it is: Reads JSON data (common for API responses) into a DataFrame.

```
python

customers = pd.read_json("customer_info.json")
```

⚠ **Warning:** Deeply nested JSON (e.g. an address object inside a customer record) won't automatically flatten — you'll need `pd.json_normalize()` for that.

`pd.read_parquet()`

What it is: Reads Parquet, a compressed, column-oriented binary format popular in big-data pipelines. **Why it matters:** Parquet files are smaller and load much faster than CSV for large datasets, and they **preserve data types** (a CSV forgets that a column was an integer; Parquet remembers).

```
python
```

```
inventory = pd.read_parquet("product_inventory.parquet")
```

🌟 **Interview Tip:** If asked "CSV vs Parquet," mention: Parquet is typed, compressed, and columnar (fast for reading specific columns); CSV is human-readable, universal, but untyped and larger.

First-Look Functions

`df.head(n)` / `df.tail(n)`

Shows the first/last `n` rows (default 5). Your very first command on any new dataset.

```
python
```

```
housing.head()           # sanity check: do the columns look right?
housing.tail(3)
```

`df.sample(n)`

Shows `n` **random** rows instead of the first ones. 💡 **Pro Tip:** `head()` can be misleading if the file is sorted (e.g. sorted by date) — `sample(10)` gives you a much more honest first impression of messy real-world rows.

```
python
```

```
banking.sample(5, random_state=42)  # random_state makes it reproducible
```

`df.info()`

Shows column names, non-null counts, data types, and memory usage — all in one call.

```
python
```

```
medical_records.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   patient_id      5000 non-null   int64
1   age             4890 non-null   float64
2   diagnosis       4975 non-null   object
3   blood_group     4700 non-null   object
4   visit_date      5000 non-null   object
5   bill_amount     4999 non-null   float64
```

This single output already tells you: `age`, `diagnosis`, `blood_group`, and `bill_amount` have missing values, and `visit_date` is still text (`object`), not a real date — two chapters' worth of work revealed in one line.

`df.describe()`

Summary statistics (count, mean, std, min, quartiles, max) for numeric columns by default.

```
python  
housing.describe()
```

Add `include='object'` to summarize text/categorical columns instead (count, unique, top value, frequency of top value):

```
python  
housing.describe(include='object')
```

⚠ **Warning:** `describe()` silently ignores non-numeric columns unless you ask for them with `include=`. A common mistake is assuming a column wasn't summarized because it "had no data," when really it was just skipped.

`df.shape`, `df.columns`, `df.index`, `df.dtypes`

Quick structural facts, all accessed as **attributes** (no parentheses):

```
python  
  
sales.shape      # (12500, 8)  -> 12,500 rows, 8 columns  
sales.columns    # Index(['order_id', 'customer_id', ...])  
sales.index      # RangeIndex(start=0, stop=12500, step=1)  
sales.dtypes     # data type of every column
```

💡 **Pro Tip:** Run `df.shape` again after every major cleaning step (dropping rows, dropping duplicates). If the row count drops more than you expect, you likely deleted good data by mistake.

Chapter 3 — Understanding Your Dataset

Before you clean anything, you need a diagnostic report of *what's actually wrong*. This chapter is that diagnostic toolkit.

Data Types Recap

Pandas dtype	Meaning	Example column
<code>int64</code>	Whole numbers	<code>employee_id</code>
<code>float64</code>	Decimal numbers (also used when a numeric column has missing values!)	<code>salary</code>
<code>object</code>	Text or mixed types	<code>customer_name</code>
<code>bool</code>	True/False	<code>is_active</code>
<code>datetime64</code>	Dates/times	<code>order_date</code>
<code>category</code>	A fixed, limited set of labels	<code>department</code>

⚠ **Warning:** A numeric column with even one missing value gets upgraded from `int64` to `float64`, because `NaN` only exists as a float in Pandas. Seeing `4.0, 5.0, NaN, 7.0` in an "age" column is your first clue there's a missing value hiding.

`df.memory_usage()`

Shows how much memory (in bytes) each column consumes.

```
python
```

```
inventory.memory_usage(deep=True)
```

`deep=True` is important for `object` columns — without it, Pandas underestimates how much memory text columns actually use. 💡 **Pro Tip:** Converting repetitive text columns (like `category`, `department`, `country`) to the `category` dtype can cut memory usage by 80–90% on large datasets — a common interview follow-up to this function.

`df.value_counts()`

Counts how often each value appears in a **Series** (one column), sorted from most to least frequent.

```
python
```

```
restaurants["rating"].value_counts()
```

```
4.0    1204
5.0     980
3.0     645
4 stars    3    <- a typo! should be 4.0
Excellent  1    <- another typo!
```

This is the #1 way to catch **inconsistent categories** before they wreck an average or a group-by. 💡 **Pro Tip:** Add `normalize=True` to get proportions instead of counts — great for quickly checking class imbalance (e.g., "are 95% of transactions non-fraud?").

```
python
```

```
transactions["is_fraud"].value_counts(normalize=True)
```

`df.unique()` and `df.nunique()`

- `unique()` — returns the actual distinct values (works on a Series, not a whole DataFrame)
- `nunique()` — returns the **count** of distinct values (works on a Series or a DataFrame, column-by-column)

```
python
```

```
employees["department"].unique()    # array(['Sales', 'sales', 'SALES ', 'HR', 'E
employees["department"].nunique()    # 5  <- but really there are only 3 real depa
```

⚠️ **Warning:** `nunique()` counting "5" here is misleading — three of those "5 departments" are the same department typed differently. High cardinality doesn't always mean genuinely diverse data; sometimes it means dirty data. This directly foreshadows **Chapter 9**.

`df.duplicated()`

Flags rows that are exact repeats of an earlier row, returning `True`/`False` for each row (doesn't remove anything by itself).

```
python
```

```
banking["is_dup"] = banking.duplicated()
banking["is_dup"].sum()      # how many duplicate rows exist
```

Covered in full, with all its parameters, in **Chapter 5**.

Putting It Together: A Quick Diagnostic Script

```
python

def quick_diagnostic(df):
    print("Shape:", df.shape)
    print("\nData types:\n", df.dtypes)
    print("\nMissing values:\n", df.isnull().sum())
    print("\nDuplicate rows:", df.duplicated().sum())
    print("\nMemory usage (MB):", df.memory_usage(deep=True).sum() / 1e6)

quick_diagnostic(customer_info)
```

 **Interview Tip:** Interviewers love asking "What's the first thing you do with a new dataset?" A function like the one above — shape, dtypes, missing values, duplicates, memory — is a strong, structured answer.

Chapter 4 — Handling Missing Values (Most Detailed Chapter)

Missing values are the single most common data quality problem you'll face — and the one with the most ways to get wrong. This chapter goes deep.

Why Missing Values Occur

- **Not collected** — a customer skipped an optional field like "middle name" or "phone number"
- **Not applicable** — a "spouse name" field is blank for unmarried customers; it's not missing, it genuinely doesn't apply
- **System/merge errors** — joining two tables where some IDs don't match leaves gaps
- **Human error** — a nurse forgot to record a patient's blood pressure
- **Sensor/device failure** — an IoT sensor drops readings during a network outage
- **Privacy redaction** — a bank intentionally blanks out sensitive fields in a shared dataset

The Three Types of Missing Data (Know This for Interviews)

Type	Meaning	Example
MCAR — Missing Completely At Random	The missingness has <i>no relationship</i> to any data, observed or not	A survey page failed to save for a random subset of respondents
MAR — Missing At Random	Missingness relates to <i>other observed</i> columns	Income is missing more often for respondents who didn't list an occupation
MNAR — Missing Not At Random	Missingness relates to the <i>missing value itself</i>	High earners are less likely to disclose their income

 **Interview Tip:** MNAR is the hardest case — the fact that data is missing is itself informative, and naive imputation (like filling with the mean) can be actively misleading. Mentioning this distinction signals strong statistical understanding.

Detecting Missing Values

`isnull()` / `isna()` (identical — `isna()` is the modern alias)

Returns a same-shaped DataFrame/Series of `True`/`False`.

python

```
medical_records.isnull().sum()           # count of missing values PER COLUMN
medical_records.isnull().sum().sum()      # total missing values in the whole dataset
medical_records.isnull().mean() * 100    # % missing per column — often more useful
```

`notnull()` / `notna()`

The opposite — flags where data **is** present.

python

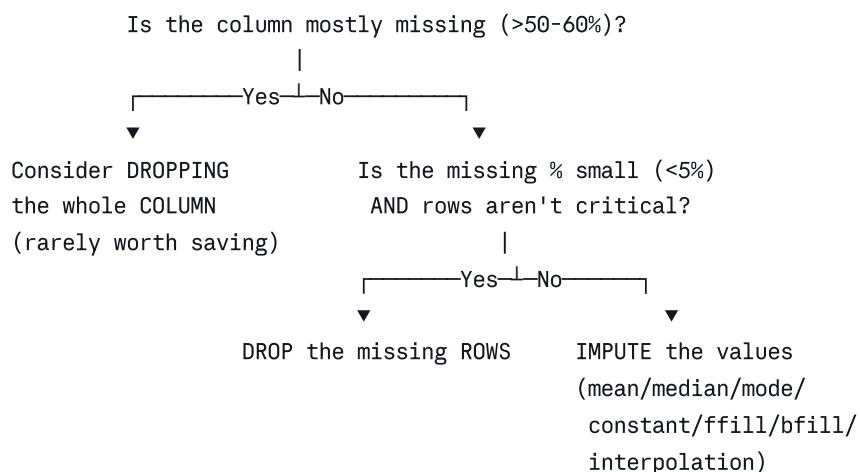
```
medical_records[medical_records["diagnosis"].notnull()] # keep only rows with a d
```

💡 **Pro Tip:** Visualize missingness before deciding what to do. A simple heatmap makes patterns (like "these three columns always go missing together") obvious in seconds:

python

```
import seaborn as sns
sns.heatmap(medical_records.isnull(), cbar=False)
```

Deciding What To Do: Delete Rows, Delete Columns, or Impute?



`dropna()` — Deleting Rows or Columns

Syntax:

python

```
df.dropna(axis=0, how='any', thresh=None, subset=None, inplace=False)
```

- `axis=0` (default) drops rows; `axis=1` drops columns
- `how='any'` drops if **any** value in the row/column is missing; `how='all'` drops only if **everything** is missing
- `subset` — only consider missing values in specific columns
- `thresh` — keep rows/columns with at least this many non-null values

When to delete rows:

- The missing rows are a small fraction of the dataset (rule of thumb: under ~5%)
- The missing values are in a critical column you can't reliably estimate (e.g. a bank transaction with no `account_id`)

python

```
# Banking Transactions: an account_id is mandatory – a transaction without one is u  
transactions = transactions.dropna(subset=["account_id"])
```

When to delete columns:

- A column is missing so much data (commonly >50-60%) that imputing it would mostly be guessing
- The column isn't useful for the analysis anyway

python

```
# Customer Information: "middle_name" is 92% empty and irrelevant to the analysis  
customers = customers.drop(columns=["middle_name"])
```

⚠ **Warning:** Dropping rows changes your sample size and can introduce bias — if higher-income customers skip the "income" field more often (MNAR), dropping those rows skews your remaining data toward lower incomes.

✗ **Common Mistake:** Calling `dropna()` with no arguments on the whole DataFrame. If even one column has a handful of missing values, `how='any'` (the default) can wipe out the majority of your rows.

Imputation Methods

Mean Imputation

Replace missing numeric values with the column average.

python

```
housing["price"] = housing["price"].fillna(housing["price"].mean())
```

- ✅ **Advantages:** Simple, preserves the sample size, keeps the overall mean unchanged
- ✗ **Disadvantages:** Shrinks variance (makes data look artificially more consistent), heavily skewed by outliers
- **Best for:** Normally distributed numeric data with few outliers (e.g. `square_footage` in Housing Prices)

Median Imputation

Replace missing numeric values with the middle value.

python

```
housing["price"] = housing["price"].fillna(housing["price"].median())
```

- ✅ **Advantages:** Robust to outliers — a few mansion listings won't drag the "typical" price up
- ✗ **Disadvantages:** Still reduces variance; ignores relationships with other columns

- **Best for:** Skewed numeric data — this is why **median** (not mean) is the standard default for things like `price`, `income`, and `bill_amount`

💡 **Pro Tip:** When in doubt between mean and median for a real-world money column, check the skew first: `df["price"].skew()`. If it's far from 0, use the median.

Mode Imputation

Replace missing categorical values with the most frequent category.

```
python
employees["department"] = employees["department"].fillna(employees["department"].mo
```

⚠️ **Warning:** `.mode()` returns a *Series* (there can be ties), so you almost always need `[0]` to grab the single most common value.

- **Best for:** Categorical columns like `department`, `blood_group`, `payment_method`

Constant Value Replacement

Replace missing values with a fixed, meaningful placeholder.

```
python
customers["referral_source"] = customers["referral_source"].fillna("Not Provided")
product_inventory["discount_pct"] = product_inventory["discount_pct"].fillna(0)
```

- **Best for:** Cases where "missing" has real meaning — a blank `referral_source` likely means "walked in / no referral," and a blank `discount_pct` likely means "no discount," which is genuinely `0`, not unknown.
- 🎯 **Interview Tip:** This is the imputation method interviewers expect you to bring up *unprompted* — it shows you think about **business meaning**, not just statistics.

Forward Fill (`ffill()`) and Backward Fill (`bfill()`)

Carries the previous (or next) valid value forward (or backward) to fill gaps.

```
python
sales_data["stock_price"] = sales_data["stock_price"].ffill() # repeat last known
sales_data["stock_price"] = sales_data["stock_price"].bfill() # pull the next kno
```

- **Best for:** Time-ordered data where values change slowly — daily stock prices, sensor readings, sequential sales logs
- ⚠️ **Warning:** Using `ffill()`/`bfill()` on data that ISN'T ordered by time (e.g. randomly shuffled customer rows) fills gaps with a value that has no real relationship to the missing row.

Interpolation

Estimates missing values using the trend between the surrounding known values.

```
python
sales_data["daily_revenue"] = sales_data["daily_revenue"].interpolate(method="linea
```

- **Best for:** Smoothly changing numeric time series — e.g. a temperature sensor with a few missing readings

- ✗ **Disadvantages:** Assumes a smooth trend; won't work well for volatile or spiky data (like daily lottery ticket sales)

Decision-Making Summary Table

Situation	Recommended approach
Numeric, normal distribution, few outliers	Mean imputation
Numeric, skewed or has outliers	Median imputation
Categorical column	Mode imputation
Missing has real business meaning ("no discount," "no referral")	Constant value
Time-ordered, slow-changing data	Forward fill / Backward fill
Time-ordered, smoothly trending numeric data	Interpolation
Column missing >50-60% of values	Drop the column
Row missing a critical identifier, and rows are few	Drop the row

🧠 **Interview Tip:** If asked "how do you handle missing data," never say just "I use `fillna()`." Walk through this decision tree instead — it shows judgment, not just syntax memorization.

Chapter 5 — Removing Duplicate Data

Detecting Duplicates: `df.duplicated()`

Syntax:

```
python
df.duplicated(subset=None, keep='first')
```

Returns `True`/`False` per row — `True` means "this row is a repeat of an earlier one."

- `subset` — check duplicates based on only certain columns (e.g. just `email`, ignoring that `signup_date` differs)
- `keep`:
 - `'first'` (default) — marks all but the *first* occurrence as duplicate
 - `'last'` — marks all but the *last* occurrence as duplicate
 - `False` — marks **every** copy (including the first) as duplicate

Example — E-commerce Orders:

```
python
orders.duplicated(subset=["order_id"]).sum() # how many order_ids repeat?
orders[orders.duplicated(subset=["order_id"], keep=False)] # see ALL copies, to i
```

Removing Duplicates: `df.drop_duplicates()`

Same parameters as `df.duplicated()`, but actually removes the rows.

```
python
```

```
orders_clean = orders.drop_duplicates(subset=["order_id"], keep="first")
```

💡 **Pro Tip:** Always inspect duplicates with `keep=False` *before* dropping them with `keep='first'`. Sometimes "duplicate" rows aren't true duplicates — e.g. two restaurant reviews from the same customer on the same day for the same restaurant could be two genuinely separate visits.

When Duplicates Are Useful (Don't Always Remove Them!)

- **Banking Transactions:** A customer genuinely buying coffee twice in one day at the same shop for the same amount is not a duplicate — it's two real transactions. Removing it by `drop_duplicates()` on the whole row would incorrectly erase real revenue.
- **Sales Data:** Repeated line items (same product, same order, different order lines) can be legitimate if a customer ordered the same item in two separate baskets.

⚠️ **Warning:** The right way to check for *true* duplicates is rarely "the whole row matches" — it's usually a specific business key (`transaction_id`, `order_id`) repeating, which signals a system or data-entry error, not real activity.

When Duplicates Should Never Be Removed

- Time-series sensor logs where identical readings in a row are a real (if boring) pattern
- Any dataset where the "duplicate" rows have different timestamps, IDs, or other columns that make them genuinely distinct events

❌ **Common Mistake:** Running `df.drop_duplicates()` with no arguments right after loading, without first checking *why* rows are duplicated. This can silently delete real, valid records.

Chapter 6 — Fixing Data Types

Wrong data types are one of the sneakiest sources of bugs — a column can *look* numeric in `head()` while actually being stored as text, and every calculation on it will fail or misbehave.

`astype()`

What it is: Force-converts a column to a specified type. **Syntax:**

```
python
df["column"] = df["column"].astype(new_type)
```

Examples:

```
python
# String -> Integer (Employee Records: "years_experience" loaded as text)
employees["years_experience"] = employees["years_experience"].astype(int)

# String -> Float (Housing Prices: "lot_size_sqft" loaded as text)
housing["lot_size_sqft"] = housing["lot_size_sqft"].astype(float)

# Object -> Boolean (Product Inventory: "in_stock" stored as "True"/"False" strings)
inventory["in_stock"] = inventory["in_stock"].astype(bool)
```

⚠ **Warning:** `astype()` crashes immediately on the first bad value (e.g. a stray `"N/A"` in a numeric column). Use `pd.to_numeric()` with `errors="coerce"` instead when the column might contain junk values.

`pd.to_numeric()`

What it is: Safely converts to a numeric type, with control over how errors are handled.

Syntax:

```
python
pd.to_numeric(series, errors='raise', downcast=None)
```

- `errors='raise'` (default) — crash on bad values
- `errors='coerce'` — turn bad values into `NaN` instead of crashing
- `errors='ignore'` — leave the column unchanged if conversion fails (deprecated in modern Pandas — avoid)

Example — Sales Data with junk values:

```
python
sales["quantity"] = pd.to_numeric(sales["quantity"], errors="coerce")
sales["quantity"].isnull().sum()  # see how many rows had unconvertible junk
```

`pd.to_datetime()`

Converts text/object columns into real datetime objects — the gateway to everything in **Chapter 13**.

```
python
orders["order_date"] = pd.to_datetime(orders["order_date"], format="%d-%m-%Y", erro
```

`pd.to_timedelta()`

Converts text into a duration (time span) rather than a fixed point in time.

```
python
support_tickets["resolution_time"] = pd.to_timedelta(support_tickets["resolution_ti
# e.g. converts "2 days 03:15:00" into a usable Timedelta for math like avg resolut
```

`convert_dtypes()`

What it is: Lets Pandas *guess* the best modern dtype for every column at once (nullable integers, strings, booleans).

```
python
employees = employees.convert_dtypes()
```

💡 **Pro Tip:** `convert_dtypes()` is a fast first pass on a freshly loaded CSV, but always double-check the result with `.dtypes` afterward — automatic guessing isn't a substitute for understanding your columns.

Categorical Conversion

For columns with a small, fixed set of repeating values, converting to `category` saves memory and speeds up group-by operations.

```
python
```

```
employees["department"] = employees["department"].astype("category")
```

Common Conversion Errors & How to Fix Them

Error / Symptom	Cause	Fix
<code>ValueError: could not convert string to float</code>	A stray non-numeric value (like <code>"N/A"</code> or <code>"\$500"</code>) in the column	Use <code>pd.to_numeric(..., errors="coerce")</code> , or clean symbols first (Chapter 12)
Dates loaded but <code>dtype</code> still shows <code>object</code>	Forgot to explicitly convert with <code>pd.to_datetime()</code>	Always convert date-looking columns explicitly — Pandas doesn't do it automatically on <code>read_csv()</code> unless you pass <code>parse_dates=</code>
<code>OutOfBoundsDatetime</code>	A date typo like year <code>2124</code> or <code>0024</code>	Inspect with <code>errors="coerce"</code> first, then decide whether to fix or drop those rows
Column of <code>"True"</code> / <code>"False"</code> text converts to <code>True</code> for everything	<code>astype(bool)</code> treats any non-empty string (including <code>"False"</code> !) as <code>True</code>	Map explicitly: <code>.map({"True": True, "False": False})</code>

Chapter 7 — Cleaning Text Data

Text columns are where inconsistency hides best — `"John"`, `"john "`, and `"JOHN"` look the same to a human skimming a spreadsheet, but are three different values to Pandas.

All of these are accessed through the `.str` accessor, applying a string operation to an entire column at once.

Case & Whitespace

Method	What it does	Example
<code>str.lower()</code>	Lowercases everything	<code>"JOHN"</code> → <code>"john"</code>
<code>str.upper()</code>	Uppercases everything	<code>"john"</code> → <code>"JOHN"</code>
<code>str.title()</code>	Capitalizes each word	<code>"john smith"</code> → <code>"John Smith"</code>
<code>str.capitalize()</code>	Capitalizes only the first letter	<code>"john smith"</code> → <code>"John smith"</code>
<code>str.strip()</code>	Removes leading/trailing whitespace	<code>" John "</code> → <code>"John"</code>
<code>str.lstrip()</code> / <code>str.rstrip()</code>	Removes whitespace from left/right only	<code>" John"</code> → <code>"John"</code>

Example — Customer Information:

```
python

customers["full_name"] = customers["full_name"].str.strip().str.title()
customers["email"] = customers["email"].str.strip().str.lower()
```

💡 **Pro Tip:** Emails should always be lowercased before deduplication or matching — `John@Gmail.com` and `john@gmail.com` are the same inbox but different strings to Pandas.

Replacing & Searching

`str.replace(pat, repl)`

```
python

# Restaurant Reviews: standardize a messy rating column
reviews["rating_text"] = reviews["rating_text"].str.replace("stars", "", regex=False)
```

`str.contains(pat)`

Returns True/False — great for filtering (paired with **Chapter 11**).

```
python

# Banking Transactions: find suspicious memo text
transactions[transactions["memo"].str.contains("test", case=False, na=False)]
```

⚠️ **Warning:** `str.contains()` raises an error on missing values unless you pass `na=False` (treat missing as "doesn't contain") — a very common runtime crash for beginners.

`str.startswith()` / `str.endswith()`

```
python

products[products["sku"].str.startswith("ELEC")]      # electronics SKUs
customers[customers["email"].str.endswith("@gmail.com")]
```

Splitting & Extracting

`str.split(pat)`

Splits a column into a list (or, with `expand=True`, into separate columns).

```
python

# Employee Records: "full_name" -> first_name, last_name
employees[["first_name", "last_name"]] = employees["full_name"].str.split(" ", n=1,
```

`str.extract(pattern)`

Pulls out substrings using a regular expression — the most powerful text-cleaning tool in Pandas.

```
python
```

```
# Extracting emails from a messy "contact_info" column
customers["email"] = customers["contact_info"].str.extract(r'([\w\.-]+@[ \w\.-]+)')

# Extracting a 10-digit phone number, ignoring dashes/spaces
customers["phone_clean"] = customers["contact_info"].str.extract(r'(\d{10})')
```

`str.len()`

Returns the length of each string — useful for spotting suspiciously short/long entries.

```
python

customers[customers["phone_clean"].str.len() != 10] # flag incomplete phone numbe
```

Worked Example — Cleaning Messy Customer Names

```
python

raw = [" john SMITH", "Mary-jane o'connor ", "AL bundy"]
names = pd.Series(raw)

names = (
    names
    .str.strip()
    .str.replace(r"\s+", " ", regex=True) # collapse multiple spaces into one
    .str.title()
)
print(names.tolist())
# ['John Smith', 'Mary-Jane O'Connor', 'Al Bundy']
```

✗ **Common Mistake:** Cleaning text with `.str.title()` alone and calling it done — it doesn't fix double spaces, leading/trailing whitespace, or punctuation. Always **strip** → **collapse spaces** → **fix case**, in that order.

🔗 **Interview Tip:** If asked to clean a text column, always mention regex-based extraction (`.str.extract()`) — it's the detail that separates "I called `.str.lower()`" answers from answers that show real regex fluency.

Chapter 8 — Handling Outliers

What Are Outliers?

An outlier is a data point that differs significantly from the rest of the dataset. Example: in a `Housing Prices` dataset where most homes sell for \$200K-\$600K, a listing at \$15,000,000 (a mansion, or a data-entry error with an extra zero) is an outlier.

Why Are They Dangerous?

- They **distort the mean** (median is much more resistant)
- They can make a model "chase" a rare extreme case instead of learning the general pattern
- They sometimes signal a **data error** (age = 250) rather than a real extreme value (a legitimate \$15M mansion) — and these two cases need completely different treatment

Method 1 — IQR (Interquartile Range)

The IQR method flags anything far outside the "normal middle 50%" of the data.

python

```
Q1 = housing["price"].quantile(0.25)
Q3 = housing["price"].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers = housing[(housing["price"] < lower_bound) | (housing["price"] > upper_bound)]
```

- **Best for:** Skewed data (income, price, bill amounts) — doesn't assume a bell-curve shape
- 🧠 **Interview Tip:** The " $1.5 \times \text{IQR}$ " rule is the classic textbook threshold — knowing it cold is a very common interview checkpoint.

Method 2 — Z-Score

Measures how many standard deviations a value is from the mean. A common threshold is $|z| > 3$.

python

```
from scipy import stats
medical_records["z_score"] = stats.zscore(medical_records["bill_amount"].dropna())
outliers = medical_records[medical_records["z_score"].abs() > 3]
```

- **Best for:** Roughly normally distributed data
- ⚠️ **Warning:** Z-score itself is calculated using the mean and standard deviation — both of which outliers distort! On heavily skewed data, IQR is usually the safer default.

Method 3 — Percentile Method

Simply treat the top/bottom X% of values as outliers, regardless of distribution shape.

python

```
lower = employees["salary"].quantile(0.01)
upper = employees["salary"].quantile(0.99)
trimmed = employees[(employees["salary"] >= lower) & (employees["salary"] <= upper)]
```

- **Best for:** Business rules like "the top and bottom 1% of transactions are always manually reviewed anyway"

`quantile()` and `clip()`

- `quantile(q)` — returns the value at percentile `q` (e.g. `0.5` = median)
- `clip(lower, upper)` — instead of removing outlier rows, **caps** them at a boundary (also called "winsorizing")

python

```
# Cap extreme restaurant bill amounts instead of deleting the rows
reviews["bill_amount"] = reviews["bill_amount"].clip(lower=0, upper=reviews["bill_a"])
```

💡 **Pro Tip:** `clip()` is often better than deleting outlier rows when you don't want to lose the rest of that row's information (e.g. the review text, the timestamp) — you just don't want one extreme number distorting an aggregate.

When to Remove Outliers vs. When to Keep Them

Keep the outlier	Remove/cap the outlier
It's a real, rare event with business value (a \$15M mansion that genuinely sold)	It's a clear data-entry error (age = 250, negative price)
You're specifically studying extremes (fraud detection, luxury market analysis)	It's distorting an average you're reporting to stakeholders
Removing it would bias the model against real edge cases	The model is very sensitive to scale (e.g. linear regression, k-means)

⚠️ **Warning:** In **fraud detection** (Banking Transactions), outliers are often *the entire point* of the analysis — removing them removes the fraud you were trying to detect. Always ask "is this outlier the signal, or the noise?" before touching it.

Chapter 9 — Cleaning Categorical Data

Categorical columns break in a very specific way: the *number* of categories looks fine, but several of them are actually the same real-world category, spelled differently.

`replace()`

Swaps specific values for others — the direct fix for typos and inconsistent labels.

```
python
# Employee Records: standardize department names
employees["department"] = employees["department"].replace({
    "sales": "Sales", "SALES ": "Sales", "Sale": "Sales",
    "HR ": "HR", "Human Resources": "HR"
})
```

`map()`

Applies a lookup dictionary (or function) to every value — great for recoding a whole column at once.

```
python
# Customer Information: standardize Yes/No style answers
customers["newsletter_signup"] = customers["newsletter_signup"].map({
    "yes": True, "Yes": True, "Y": True, "1": True,
    "no": False, "No": False, "N": False, "0": False
})
```

⚠️ **Warning:** Any value in the column that ISN'T a key in your dictionary gets converted to `NaN` by `map()`. Always run `.unique()` first to see every value you need to cover, or use `replace()` instead if you only want to fix *some* values and leave the rest untouched.

`rename()`

Renames columns (or index labels) — not values, but the *labels themselves*.

```
python
```

```
sales_data = sales_data.rename(columns={"cust_id": "customer_id", "amt": "amount"})
```

`rename_axis()`

Renames the *name* of the index or columns axis itself (not individual labels).

python

```
sales_data = sales_data.rename_axis("transaction_number")
```

`pd.Categorical()`

Explicitly defines a column as categorical, optionally with a meaningful **order**.

python

```
inventory["stock_level"] = pd.Categorical(
    inventory["stock_level"],
    categories=["Out of Stock", "Low", "Medium", "High"],
    ordered=True
)
inventory.sort_values("stock_level") # now sorts logically, not alphabetically
```

💡 **Pro Tip:** Without `ordered=True` and explicit `categories=`, sorting `"Low" / "Medium" / "High"` alphabetically gives the nonsensical order High, Low, Medium. This is the single biggest reason to use `pd.Categorical()`.

Real Examples of Category Inconsistencies

Raw values found in the column	Real categories	Dataset
<code>"Y", "y", "Yes", "1", "true"</code>	<code>Yes</code>	Customer Information
<code>"M", "Male", "male ", "MALE"</code>	<code>Male</code>	Medical Records
<code>"US", "USA", "United States", "U.S.A."</code>	<code>United States</code>	E-commerce Orders
<code>"5 star", "Five Stars", "5.0", "Excellent"</code>	<code>5</code>	Restaurant Reviews

A Practical Cleanup Pattern

python

```
# Step 1: see everything you're dealing with
print(customers["country"].unique())

# Step 2: lowercase + strip to shrink accidental duplicates first
customers["country"] = customers["country"].str.strip().str.upper()

# Step 3: map remaining real inconsistencies explicitly
customers["country"] = customers["country"].replace({
    "USA": "US", "UNITED STATES": "US", "U.S.A.": "US",
    "UK": "GB", "UNITED KINGDOM": "GB"
})
```

🌀 **Interview Tip:** This "inspect → normalize case/whitespace → explicitly map remaining typos" order is worth stating explicitly — it shows a systematic process rather than ad-hoc fixing.

Chapter 10 — Column Operations

Adding, Removing, and Reordering Columns

`drop()`

```
python

housing = housing.drop(columns=["listing_agent_notes"]) # drop by name
housing = housing.drop(index=[0, 1])                  # drop specific rows by 1
```

`insert()`

Inserts a new column at a specific position (default methods add to the end).

```
python

employees.insert(loc=1, column="full_name", value=employees["first_name"] + " " + e
```

`assign()`

Creates one or more new columns **without modifying the original DataFrame** (returns a new one) — great for chaining.

```
python

sales_data = sales_data.assign(
    total_price=lambda d: d["quantity"] * d["unit_price"],
    is_bulk_order=lambda d: d["quantity"] > 50
)
```

`copy()`

Creates an independent copy so edits don't silently affect the original DataFrame.

```
python

backup = customers.copy()
```

⚠ **Warning:** `(filtered = customers[customers["country"] == "US"])` followed by editing `(filtered)` can trigger Pandas' infamous `(SettingWithCopyWarning)`. Use `(.copy())` explicitly when you intend to keep editing a filtered subset.

`pop()`

Removes a column **and returns it** in one step.

```
python

discount_col = product_inventory.pop("legacy_discount_flag")
```

Renaming, Indexing, and Sorting

`(rename())` — see Chapter 9 for the categorical-label use case; also works for straightforward column renaming as shown above.

`(set_index())` / `(reset_index())`

```
python
```

```
employees = employees.set_index("employee_id")    # make employee_id the row label
employees = employees.reset_index()               # move it back to being a normal
```

💡 **Pro Tip:** `reset_index(drop=True)` throws away the old index entirely instead of turning it into a column — use this after filtering/sorting when the old row numbers are meaningless.

`sort_values()` / `sort_index()`

```
python

housing.sort_values("price", ascending=False)      # highest price first
housing.sort_values(["city", "price"], ascending=[True, False]) # sort within groups
housing.sort_index()                             # back to original
```

Worked Example — Employee Records Cleanup

```
python

employees = (
    employees
    .drop(columns=["internal_notes"])
    .rename(columns={"dept": "department", "sal": "salary"})
    .assign(annual_bonus=lambda d: d["salary"] * 0.1)
    .sort_values("salary", ascending=False)
    .reset_index(drop=True)
)
```

Chapter 11 — Filtering Data

`loc` and `iloc`

Already covered in depth in the companion Pandas handbook — the short version:

- `loc[row_label, col_label]` — select by **name**
- `iloc[row_position, col_position]` — select by **integer position**

```
python

housing.loc[housing["price"] > 500000, ["address", "price"]]
housing.iloc[0:5, 0:3]
```

`query()`

Filters using a readable string expression instead of bracket-heavy boolean masks.

```
python

housing.query("price > 500000 and bedrooms >= 3")
```

💡 **Pro Tip:** `query()` is especially handy for long, multi-condition filters — no repeated `df[...]` clutter, and column names with spaces can be wrapped in backticks:

```
df.query("`sale price` > 500000")
```

`isin()`

Keeps rows whose value is in a given list.

```
python
```

```
employees[employees["department"].isin(["Sales", "Marketing"])]
```

`between()`

Keeps numeric values inside an inclusive range.

```
python
```

```
medical_records[medical_records["age"].between(18, 65)]
```

`where()`

Keeps values that match a condition; **replaces** the rest with `NaN` (or a custom value) instead of dropping rows — the DataFrame keeps its original shape.

```
python
```

```
sales_data["quantity"] = sales_data["quantity"].where(sales_data["quantity"] >= 0,
```

`mask()`

The exact opposite of `where()` — replaces values **where the condition is True**.

```
python
```

```
# Any negative age is clearly an error -> mask it to missing for later imputation  
medical_records["age"] = medical_records["age"].mask(medical_records["age"] < 0)
```

Practical Examples

Removing invalid records (Medical Records):

```
python
```

```
medical_records = medical_records[(medical_records["age"] >= 0) & (medical_records["
```

Selecting ranges (Housing Prices):

```
python
```

```
mid_range_homes = housing[housing["price"].between(200000, 600000)]
```

Combining conditions (E-commerce Orders):

```
python
```

```
flagged_orders = orders[  
    (orders["order_total"] > 1000) &  
    (orders["shipping_country"] != orders["billing_country"]) &  
    (~orders["status"].isin(["cancelled", "refunded"]))  
]
```

⚠ **Warning:** Every individual condition inside a combined filter needs its own parentheses — `df[a & b]` without parentheses around `a` and `b` throws an error or silently misevaluates, because of Python's operator precedence.

 **Interview Tip:** `where()`/`mask()` vs. boolean filtering (`df[condition]`) is a common "what's the difference" question: filtering **removes rows** (changes shape); `where()`/`mask()` **replaces values** (keeps shape). Being able to state that one-liner cleanly is worth rehearsing.

Chapter 12 — Cleaning Numeric Values

Numbers stored as text with symbols attached are one of the most common real-world messes — a price column that *looks* numeric in Excel often loads into Pandas as pure text.

The Core Pattern: Strip, Then Convert

```
python

# Housing Prices: "$2,500,000" -> 2500000
housing["price"] = (
    housing["price"]
    .astype(str)
    .str.replace(r"[$,]", "", regex=True)
    .astype(float)
)
```

Common Symbols to Remove, By Dataset

Symbol / unit	Example raw value	Dataset	Cleaning approach
\$	"\$2,500"	Housing Prices, E-commerce Orders	<code>.str.replace(r"[\$,]", "", regex=True)</code>
₹	"₹45,000"	Banking Transactions	<code>.str.replace(r"[₹,]", "", regex=True)</code>
%	"15%"	Sales Data (discount)	<code>.str.rstrip("%").astype(float) / 100</code>
kg	"2.5kg"	Product Inventory	<code>.str.replace("kg", "", regex=False).astype(float)</code>
cm	"180cm"	Medical Records (height)	<code>.str.replace("cm", "", regex=False).astype(float)</code>
commas as thousand separators	"1,000,000"	Sales Data	<code>.str.replace(",", "", regex=False)</code>
stray spaces	" 2500 "	Employee Records (salary)	<code>.str.strip()</code> before converting

Full Worked Example — Sales Data Discount Column

```
python
```

```
raw = pd.Series(["10%", "5 %", "0.15", "20", None])

cleaned = (
    raw
    .astype(str)
    .str.replace("%", "", regex=False)
    .str.strip()
    .replace("None", np.nan)
    .astype(float)
)

# Now need one more decision: is "20" meant to be 20% or already 0.20?
# This is a business-logic question, not a syntax question -- always confirm
# with a data dictionary or the source system rather than guessing.
```

⚠ **Warning:** Mixed formats (`"0.15"` meaning 15%, right next to `"20"` meaning 20%) are extremely common when a field was manually entered by different people. Cleaning the *symbols* is easy; cleaning the *meaning* requires checking with whoever owns the data.

General Approach for Any "Numbers with Units" Column

1. Convert the column to string with `.astype(str)` (handles a mix of already-numeric and text values safely)
2. Use `.str.replace()` with a regex character class covering every symbol you've seen (check with `.unique()` first!)
3. Strip whitespace
4. Convert with `pd.to_numeric(errors="coerce")` instead of `astype(float)` if you're not 100% sure every row is now clean — `coerce` turns any leftover junk into `NaN` instead of crashing your whole script
5. Check `.isnull().sum()` afterward to see how many rows failed conversion, and investigate those specifically

💡 **Pro Tip:** Always run `df["column"].unique()` (or at least a sample) BEFORE writing your regex — guessing which symbols exist and missing one is the #1 reason numeric cleaning scripts still leave `NaN`s behind.

Chapter 13 — Cleaning Dates

`pd.to_datetime()` Revisited

```
python

orders["order_date"] = pd.to_datetime(orders["order_date"], errors="coerce")
```

Handling Different Formats in the Same Column

Real datasets often mix `MM/DD/YYYY` and `DD/MM/YYYY` in the same column (classic in Sales Data exported from different regional systems).

```
python

# Explicit format = fast AND unambiguous -- always prefer this when you know the fo
orders["order_date"] = pd.to_datetime(orders["order_date"], format="%d/%m/%Y", erro

# Mixed unknown formats: let Pandas infer per-row (slower, but works on messy real
orders["order_date"] = pd.to_datetime(orders["order_date"], format="mixed", errors=
```

⚠ **Warning:** `pd.to_datetime("01/02/2024")` **silently** assumes `MM/DD/YYYY` by default and returns January 2nd — if your data is actually `DD/MM/YYYY` (February 1st), you get a wrong-but-valid-looking date with **no error at all**. Always confirm the source format instead of trusting Pandas' default guess.

Extracting Date Parts

```
python

orders["year"] = orders["order_date"].dt.year
orders["month"] = orders["order_date"].dt.month
orders["day"] = orders["order_date"].dt.day
orders["weekday"] = orders["order_date"].dt.weekday      # 0 = Monday
orders["weekday_name"] = orders["order_date"].dt.day_name()
transactions["hour"] = transactions["timestamp"].dt.hour
```

Invalid Dates

```
python

# February 30th, year 0, or plain gibberish text all become NaT (Not a Time) with e
orders["order_date"] = pd.to_datetime(orders["order_date"], errors="coerce")
invalid_dates = orders[orders["order_date"].isnull()]
```

💡 **Pro Tip:** `NaT` is the datetime version of `NaN` — it behaves the same way with `isnull()`, `dropna()`, and `fillna()`, so all your Chapter 4 missing-value techniques apply directly to broken dates too.

Time Zones

```
python

# Banking Transactions: localize a naive timestamp to a specific timezone
transactions["timestamp"] = transactions["timestamp"].dt.tz_localize("Asia/Kolkata")

# Convert to a different timezone for a global reporting dashboard
transactions["timestamp_utc"] = transactions["timestamp"].dt.tz_convert("UTC")
```

⚠ **Warning:** Comparing a timezone-aware timestamp column to a timezone-naive one raises an error. If you're merging datasets from different regional systems, standardize every timestamp to one timezone (usually UTC) early.

Date Parsing Errors Checklist

Symptom	Likely cause	Fix
Column stays <code>object</code> after <code>read_csv()</code>	Didn't pass <code>parse_dates=</code> or convert explicitly	<code>pd.to_datetime()</code>
Dates look "shifted" by a month/day	Wrong assumed format (DD/MM vs MM/DD)	Pass explicit <code>format=</code>
<code>OutOfBoundsDatetime</code>	Typo year (<code>2124</code> , <code>0024</code>)	<code>errors="coerce"</code> , then inspect/fix or drop
<code>TypeError</code> comparing dates	Mixing timezone-aware and naive timestamps	Standardize timezones first

Chapter 14 — Combining Cleaning Techniques (Full Walkthrough)

This chapter takes one messy dataset — **Employee Records** — from raw to fully clean, explaining every decision along the way. This is the chapter to re-read right before a take-home data-cleaning assignment.

The Raw Dataset

```
python

import pandas as pd
import numpy as np

employees = pd.DataFrame({
    "emp_id": [101, 102, 103, 104, 104, 105, 106],
    "full_name": [" Ravi Kumar", "ANITA Singh ", "raj Patel", "Sunita Rao", "Sunita",
    "department": ["Sales", "sales ", "HR", "Engineering", "Engineering", "HR", "en",
    "salary": ["55000", "62,000", "N/A", "78000", "78000", "45000", "82000"],
    "joining_date": ["12/01/2021", "2021-03-15", "01-07-2019", "invalid", "invalid"],
    "age": [29, 34, -5, 41, 41, 200, 37],
})
print(employees)
```

BEFORE — What's Wrong

Column	Problem
emp_id	Row 3 and 4 are duplicate employee records
full_name	Leading/trailing spaces, inconsistent capitalization, one missing value
department	"Sales", "sales ", "eng" are inconsistent labels for 2 real departments
salary	Stored as text, has commas, and a "N/A" placeholder
joining_date	Three different date formats, plus literal "invalid" text
age	A negative value and an impossible value (200)

↓ Cleaning Step 1 — Remove Duplicate Rows

```
python

employees = employees.drop_duplicates(subset=["emp_id"], keep="first")
```

Why first? Fixing text/types on a row you're about to delete anyway is wasted work — always deduplicate early.

↓ Cleaning Step 2 — Clean Text Columns

```
python
```

```

employees["full_name"] = (
    employees["full_name"]
    .str.strip()
    .str.replace("\s+", " ", regex=True)
    .str.title()
)
employees["department"] = (
    employees["department"]
    .str.strip()
    .str.lower()
    .replace({"sales": "Sales", "hr": "HR", "eng": "Engineering", "engineering": "E

```

↓ Cleaning Step 3 — Fix Data Types

```

python

employees["salary"] = (
    employees["salary"]
    .replace("N/A", np.nan)
    .astype(str)
    .str.replace(",", "", regex=False)
)
employees["salary"] = pd.to_numeric(employees["salary"], errors="coerce")

employees["joining_date"] = pd.to_datetime(employees["joining_date"], errors="coerc

```

↓ Cleaning Step 4 — Fix Impossible Values

```

python

# Age can't be negative or over ~110 -- mask them to missing, to fix with imputatio
employees["age"] = employees["age"].mask((employees["age"] < 0) | (employees["age"]

```

↓ Cleaning Step 5 — Handle Missing Values

```

python

employees["full_name"] = employees["full_name"].fillna("Unknown")
employees["salary"] = employees["salary"].fillna(employees["salary"].median())
employees["age"] = employees["age"].fillna(employees["age"].median())
# joining_date left as NaT deliberately -- a wrong "guessed" join date could break

```

Final Clean Dataset

```

python

print(employees)

```

	emp_id	full_name	department	salary	joining_date	age
0	101	Ravi Kumar	Sales	55000.0	2021-01-12	29.0
1	102	Anita Singh	Sales	62000.0	2021-03-15	34.0
2	103	Raj Patel	HR	63500.0	2019-01-07	34.0
3	104	Sunita Rao	Engineering	78000.0	NaT	41.0
5	105	Unknown	HR	45000.0	2020-09-05	34.0
6	106	Vikram Shah	Engineering	82000.0	2022-11-23	37.0

 **Interview Tip:** When asked to "walk through how you'd clean a dataset," this exact order — **duplicates** → **text** → **types** → **impossible values** → **missing values** — is a strong, defensible structure. Explain *why* each step comes before the next (e.g. "I dedupe first so I'm not wasting cleaning effort on rows I'll delete anyway").

Chapter 15 — Pandas Data Cleaning Cheat Sheet

Organized by category for fast lookup. Every function here has appeared earlier in this guide with a full explanation — this chapter is the compressed, syntax-first version.

Loading & First Look

Function	Purpose	Syntax	Example	Important Notes
<code>pd.read_csv()</code>	Load a CSV file	<code>pd.read_csv(path, na_values=[...])</code>	<code>pd.read_csv("sales.csv")</code>	Pass <code>na_values</code> for custom missing markers
<code>pd.read_excel()</code>	Load an Excel file	<code>pd.read_excel(path, sheet_name=)</code>	<code>pd.read_excel("hr.xlsx")</code>	<code>sheet_name=None</code> loads all sheets
<code>pd.read_json()</code>	Load JSON	<code>pd.read_json(path)</code>	<code>pd.read_json("orders.json")</code>	Nested JSON needs <code>json_normalize()</code>
<code>pd.read_parquet()</code>	Load Parquet	<code>pd.read_parquet(path)</code>	<code>pd.read_parquet("inv.parquet")</code>	Preserves dtypes; fast for big data
<code>df.head()</code> / <code>df.tail()</code>	Preview rows	<code>df.head(n=5)</code>	<code>df.head(10)</code>	First check on any dataset
<code>df.sample()</code>	Random rows	<code>df.sample(n)</code>	<code>df.sample(5, random_state=1)</code>	Less biased preview than <code>head()</code>
<code>df.info()</code>	Structure summary	<code>df.info()</code>	<code>df.info()</code>	Shows dtypes + non-null counts
<code>df.describe()</code>	Summary stats	<code>df.describe(include=)</code>	<code>df.describe(include="object")</code>	Numeric only by default
<code>df.shape</code>	Row/col count	<code>df.shape</code>	<code>(1000, 8)</code>	Attribute, no <code>()</code>
<code>df.columns</code>	Column names	<code>df.columns</code>	<code>Index([...])</code>	Attribute, no <code>()</code>
<code>df.dtypes</code>	Column types	<code>df.dtypes</code>	—	Watch for <code>object</code> on dates/numbers

Understanding the Data

Function	Purpose	Syntax	Example	Important Notes
<code>df.memory_usage()</code>	Memory per column	<code>df.memory_usage(deep=True)</code>	—	Use <code>deep=True</code> for text columns
<code>df.value_counts()</code>	Frequency per value	<code>series.value_counts(normalize=)</code>	<code>df["dept"].value_counts()</code>	Great for spotting typo categories
<code>df.unique()</code>	Distinct values	<code>series.unique()</code>	<code>df["city"].unique()</code>	Series only
<code>df.nunique()</code>	Count of distinct values	<code>df.nunique()</code>	<code>df["city"].nunique()</code>	High count can mean dirty data
<code>df.duplicated()</code>	Flag duplicate rows	<code>df.duplicated(subset=, keep=)</code>	<code>df.duplicated(keep=False)</code>	Doesn't remove, only flags

Missing Values

Function	Purpose	Syntax	Example	Important Notes
<code>isnull()/isna()</code>	Flag missing	<code>df.isnull()</code>	<code>df.isnull().sum()</code>	Identical function
<code>notnull()/notna()</code>	Flag present	<code>df.notnull()</code>	<code>df[df["x"].notnull()]</code>	Opposite of <code>isnull()</code>
<code>dropna()</code>	Remove missing rows/cols	<code>df.dropna(axis=, how=, subset=)</code>	<code>df.dropna(subset=["id"])</code>	<code>how='any'</code> is the default. <code>how='all'</code> is risky on wide data
<code>fillna()</code>	Fill missing values	<code>df.fillna(value)</code>	<code>df["x"].fillna(df["x"].median())</code>	Choose mean/median/mode deliberately
<code>interpolate()</code>	Estimate via trend	<code>series.interpolate(method=)</code>	<code>df["x"].interpolate()</code>	Best for smooth trends
<code>ffill()</code>	Fill from previous value	<code>series.ffill()</code>	<code>df["price"].ffill()</code>	Needs meaningful previous values
<code>bfill()</code>	Fill from next value	<code>series.bfill()</code>	<code>df["price"].bfill()</code>	Same caveat as <code>ffill()</code>

Duplicates & Data Types

Function	Purpose	Syntax	Example	Important Notes
<code>drop_duplicates()</code>	Remove duplicate rows	<code>df.drop_duplicates(subset=, keep=)</code>	<code>df.drop_duplicates(subset=["id"])</code>	Check with <code>keep=False</code> first
<code>astype()</code>	Force type conversion	<code>series.astype(type)</code>	<code>df["age"].astype(int)</code>	Crashes on bad values
<code>pd.to_numeric()</code>	Safe numeric conversion	<code>pd.to_numeric(series, errors=)</code>	<code>pd.to_numeric(df["x"], errors="coerce")</code>	<code>coerce</code> turns junk into <code>NaN</code>
<code>pd.to_datetime()</code>	Convert to datetime	<code>pd.to_datetime(series, format=, errors=)</code>	<code>pd.to_datetime(df["d"], errors="coerce")</code>	Default date order guess can be wrong
<code>pd.to_timedelta()</code>	Convert to duration	<code>pd.to_timedelta(series)</code>	<code>pd.to_timedelta(df["duration"])</code>	For spans, not fixed points in time
<code>convert_dtypes()</code>	Auto-guess best dtypes	<code>df.convert_dtypes()</code>	<code>df.convert_dtypes()</code>	Always verify with <code>.dtypes</code> after

Text Cleaning (.str)

Function	Purpose	Syntax	Example	Important Notes
<code>str.lower()</code> / <code>str.upper()</code>	Change case	<code>series.str.lower()</code>	<code>df["email"].str.lower()</code>	Standard before deduping emails
<code>str.title()</code> / <code>str.capitalize()</code>	Capitalize words	<code>series.str.title()</code>	<code>df["name"].str.title()</code>	Doesn't fix spacing on own
<code>str.strip()</code> / <code>lstrip()</code> / <code>rstrip()</code>	Remove whitespace	<code>series.str.strip()</code>	<code>df["name"].str.strip()</code>	Do this before comparing values
<code>str.replace()</code>	Replace substrings	<code>series.str.replace(pat, repl, regex=)</code>	<code>df["x"].str.replace("\$", "", regex=False)</code>	Set <code>regex=False</code> for literal symbols
<code>str.contains()</code>	Check substring presence	<code>series.str.contains(pat, na=)</code>	<code>df["memo"].str.contains("test", na=False)</code>	Always pass <code>na=False</code>
<code>str.startswith()</code> / <code>endswith()</code>	Check prefix/suffix	<code>series.str.startswith(pat)</code>	<code>df["sku"].str.startswith("ELEC")</code>	—
<code>str.split()</code>	Split into parts	<code>series.str.split(pat, expand=)</code>	<code>df["name"].str.split(" ", expand=True)</code>	<code>expand=True</code> for separate columns
<code>str.extract()</code>	Regex extraction	<code>series.str.extract(pattern)</code>	<code>df["x"].str.extract(r'(\d{10})')</code>	Most powerful text-cleaning tool
<code>str.len()</code>	String length	<code>series.str.len()</code>	<code>df["phone"].str.len()</code>	Good for spotting malformed entries

Outliers

Function	Purpose	Syntax	Example	Important Notes
<code>quantile()</code>	Get percentile value	<code>series.quantile(q)</code>	<code>df["price"].quantile(0.75)</code>	Basis for IQR method
<code>clip()</code>	Cap values at bounds	<code>series.clip(lower=, upper=)</code>	<code>df["x"].clip(0, 1000)</code>	Alternative to deleting outlier rows

Categorical Cleaning

Function	Purpose	Syntax	Example	Important Notes
<code>replace()</code>	Fix specific values	<code>series.replace({old: new})</code>	<code>df["dept"].replace({"sales": "Sales"})</code>	Leaves unmapped values untouched
<code>map()</code>	Recode via lookup	<code>series.map({...})</code>	<code>df["flag"].map({"Y": True, "N": False})</code>	Unmapped values become <code>NaN</code>
<code>rename()</code>	Rename columns/index	<code>df.rename(columns={...})</code>	<code>df.rename(columns={"cust": "customer"})</code>	For labels, not values
<code>rename_axis()</code>	Rename the axis itself	<code>df.rename_axis("name")</code>	—	Renames the axis label, not columns
<code>pd.Categorical()</code>	Define ordered categories	<code>pd.Categorical(series, categories=, ordered=)</code>	See Chapter 9	Enables logical (not alphabetical) sorting

Column & Row Operations

Function	Purpose	Syntax	Example	Important Notes
<code>drop()</code>	Remove columns/rows	<code>df.drop(columns=, index=)</code>	<code>df.drop(columns=["notes"])</code>	—
<code>insert()</code>	Add column at position	<code>df.insert(loc, column, value)</code>	<code>df.insert(1, "full_name", ...)</code>	—
<code>assign()</code>	Add columns, chainable	<code>df.assign(new_col=lambda d: ...)</code>	See Chapter 10	Returns a new DataFrame
<code>copy()</code>	Independent duplicate	<code>df.copy()</code>	<code>backup = df.copy()</code>	Avoids <code>SettingWithCopyWarning</code>
<code>pop()</code>	Remove & return column	<code>df.pop(column)</code>	<code>col = df.pop("temp")</code>	—
<code>set_index()</code> / <code>reset_index()</code>	Manage the row index	<code>df.set_index(col)</code>	<code>df.reset_index(drop=True)</code>	<code>drop=True</code> discards old index
<code>sort_values()</code> / <code>sort_index()</code>	Sort rows	<code>df.sort_values(by, ascending=)</code>	<code>df.sort_values("price", ascending=False)</code>	Can sort by multiple columns

Filtering

Function	Purpose	Syntax	Example	Important Notes
<code>loc</code>	Label-based selection	<code>df.loc[rows, cols]</code>	<code>df.loc[df["x"]>5, "y"]</code>	Range end is inclusive
<code>iloc</code>	Position-based selection	<code>df.iloc[rows, cols]</code>	<code>df.iloc[0:5, 0:2]</code>	Range end is exclusive
<code>query()</code>	String-expression filter	<code>df.query("expr")</code>	<code>df.query("price > 500000")</code>	Cleaner for multi-condition filters
<code>isin()</code>	Membership filter	<code>series.isin([...])</code>	<code>df["dept"].isin(["HR", "Sales"])</code>	—
<code>between()</code>	Range filter	<code>series.between(a, b)</code>	<code>df["age"].between(18, 65)</code>	Inclusive on both ends
<code>where()</code>	Keep-or-replace, same shape	<code>series.where(cond, other=)</code>	<code>df["q"].where(df["q"]>=0, 0)</code>	Keeps shape (unlike boolean filtering)
<code>mask()</code>	Opposite of <code>where()</code>	<code>series.mask(cond, other=)</code>	<code>df["age"].mask(df["age"]<0)</code>	Replaces where condition is True

Dates

Function	Purpose	Syntax	Example	Important Notes
<code>dt.year</code> / <code>.month</code> / <code>.day</code>	Extract date part	<code>series.dt.year</code>	<code>df["d"].dt.year</code>	Requires datetime dtype first
<code>dt.weekday</code> / <code>.day_name()</code>	Day of week	<code>series.dt.day_name()</code>	<code>df["d"].dt.day_name()</code>	<code>weekday</code> : 0 = Monday
<code>dt.hour</code>	Extract hour	<code>series.dt.hour</code>	<code>df["ts"].dt.hour</code>	For datetime columns with time
<code>dt.tz_localize()</code> / <code>.tz_convert()</code>	Time zone handling	<code>series.dt.tz_localize(tz)</code>	<code>df["ts"].dt.tz_localize("UTC")</code>	Can't compare aware vs. naive directly

Exporting Clean Data

Function	Purpose	Syntax	Example	Important Notes
<code>to_csv()</code>	Save as CSV	<code>df.to_csv(path, index=False)</code>	<code>df.to_csv("clean.csv", index=False)</code>	<code>index=False</code> avoids a stray column
<code>to_excel()</code>	Save as Excel	<code>df.to_excel(path, index=False)</code>	<code>df.to_excel("clean.xlsx", index=False)</code>	—
<code>to_parquet()</code>	Save as Parquet	<code>df.to_parquet(path)</code>	<code>df.to_parquet("clean.parquet")</code>	Preserves dtypes for next load

💡 **Pro Tip:** Bookmark this chapter. In real projects and take-home assessments, this table covers the vast majority of what you'll actually type.

Chapter 16 — Common Beginner Mistakes

1. Deleting Too Many Rows

Why it happens: Calling `dropna()` with no arguments wipes any row with even one missing value across dozens of columns. **How to avoid it:** Use `subset=` to target only the columns that truly matter, and always check `df.shape` before and after.

2. Replacing NaN Incorrectly

Why it happens: Filling a categorical column's missing values with `0`, or a numeric column with the string `"Unknown"` — mismatched types that quietly break later calculations. **How to avoid it:** Match the fill value's type to the column's type, and re-check `.dtypes` afterward.

3. Wrong Data Type Conversion

Why it happens: Using `astype(int)` on a column that still has missing values (`NaN` can't become an `int`), causing a crash — or converting `astype(bool)` on text (`"False"` becomes `True`, because any non-empty string is truthy!). **How to avoid it:** Handle missing values *before* converting types; for booleans stored as text, always use an explicit `.map()`.

4. Using `inplace=True` Incorrectly

Why it happens: Beginners assume `inplace=True` is required, or chain it in a way that silently returns `None`.

```
python
df = df.dropna(inplace=True)  # ❌ df is now None!
```

How to avoid it: Prefer reassignment (`df = df.dropna()`) over `inplace=True` — it's more predictable and chains cleanly with other operations.

5. Chained Indexing

```
python
df[df["age"] > 30]["salary"] = 0  # ❌ may silently fail to update the real DataFrame
```

Why it happens: The first `[...]` creates a temporary copy; editing it doesn't touch the original. **How to avoid it:** Use `.loc[]` in a single step: `df.loc[df["age"] > 30, "salary"] =`

0).

6. Forgetting to Check Duplicates

Why it happens: Assuming a "clean-looking" dataset has no repeats, especially after a merge or an API retry. **How to avoid it:** Make `df.duplicated().sum()` a standard part of your first-look checklist (Chapter 3), every single time.

7. Editing the Original DataFrame Accidentally

```
python

filtered = df[df["country"] == "US"]
filtered["flag"] = True # ⚠️ SettingWithCopyWarning -- may or may not affect df
```

Why it happens: A filtered DataFrame can be a view into the original, not a true independent copy — Pandas' own behavior here is intentionally ambiguous for performance reasons. **How to avoid it:** Call `.copy()` explicitly right after filtering if you intend to modify the result: `filtered = df[df["country"] == "US"].copy()`.

8. Cleaning Text But Forgetting Whitespace

Why it happens: Calling `.str.lower()` and assuming values now match, without also stripping leading/trailing spaces. **How to avoid it:** Always chain `.str.strip()` alongside case normalization.

9. Trusting `pd.to_datetime()`'s Default Format Guess

Why it happens: `01/02/2024` silently becomes January 2nd by default, with no error, even if the source data means February 1st. **How to avoid it:** Always pass an explicit `format=` when you know the source convention.

10. Overwriting the Raw Data File

Why it happens: Saving cleaned output over the original `data.csv`, losing the ability to re-check or redo cleaning decisions later. **How to avoid it:** Always save cleaned data under a new name, like `data_clean.csv`, and treat the raw file as read-only.

Chapter 17 — Interview Questions

Beginner (20)

1. What is data cleaning, and why is it important? Data cleaning is the process of fixing or removing incorrect, missing, duplicate, or inconsistent data so it's trustworthy for analysis. It matters because every downstream report or model inherits whatever quality problems exist in the raw data — "garbage in, garbage out."

2. What's the difference between `isnull()` and `isna()`? They are identical — `isna()` is simply the more modern alias. Both return a same-shaped True/False mask marking missing values.

3. How do you check for missing values in a DataFrame? `df.isnull().sum()` gives the count per column; `df.isnull().sum().sum()` gives the total across the whole DataFrame; `df.isnull().mean() * 100` gives the percentage per column.

4. What's the difference between `dropna()` and `fillna()`? `dropna()` removes rows or columns containing missing values; `fillna()` replaces missing values with something specific (a constant, mean, median, mode, or another method) while keeping the row.

5. When would you drop a column entirely instead of imputing it? When a very large share of its values are missing (commonly over 50–60%), making imputation mostly guesswork, or when the column isn't relevant to the analysis anyway.

6. What does `duplicated()` return? A Series of True/False values, one per row, where True means that row is a repeat of an earlier occurrence (or, with `keep=False`, every occurrence of a repeated row).

7. What's the difference between `keep='first'` and `keep='last'` in `drop_duplicates()`? `keep='first'` retains the first occurrence of a duplicated row and marks later ones as duplicates; `keep='last'` does the reverse, retaining the last occurrence.

8. Why might a numeric-looking column load as `object` dtype? Usually because it contains a symbol (like `$` or `,`), a placeholder string (`"N/A"`), or inconsistent formatting that prevents Pandas from recognizing it as purely numeric.

9. What's the difference between `astype()` and `pd.to_numeric()`? `astype()` converts directly and crashes on the first invalid value; `pd.to_numeric()` offers `errors="coerce"`, which turns invalid values into `NaN` instead of crashing.

10. How do you remove leading and trailing whitespace from a text column? `df["col"] = df["col"].str.strip()`.

11. What does `value_counts()` do, and why is it useful for cleaning? It counts how many times each unique value appears in a column, sorted by frequency. It's useful because it quickly surfaces inconsistent categories (typos, casing differences) hiding among common values.

12. What is the difference between `unique()` and `nunique()`? `unique()` returns the actual array of distinct values; `nunique()` returns the count of how many distinct values there are.

13. What does `loc` do differently from `iloc`? `loc` selects by label/name (and its range slicing includes the end label); `iloc` selects by integer position (and its range slicing excludes the end position, like standard Python indexing).

14. Why does `df[df["age"] > 18 and df["city"] == "X"]` raise an error? Because Python's `and` expects a single True/False value, but a Pandas condition produces a whole Series of them. Use `&` instead, with each condition wrapped in parentheses.

15. What is an outlier? A data point that differs significantly from the rest of the dataset — it may be a genuine rare extreme value or a data-entry error, and telling those two apart is the real skill.

16. What's the IQR method for detecting outliers? Compute Q1 and Q3 (25th and 75th percentiles), find the IQR = Q3 – Q1, and flag anything below `Q1 - 1.5*IQR` or above `Q3 + 1.5*IQR` as an outlier.

17. What does `pd.to_datetime()` do, and why might it silently give a wrong answer? It converts text/numeric values into real datetime objects. It can silently misparse ambiguous dates (like `01/02/2024`) because it assumes a default format (commonly month/day/year) unless you specify `format=` explicitly.

18. Why should you avoid overwriting your raw data file when saving cleaned data? So you always have the original to fall back on if a cleaning decision turns out wrong, or if you need to redo the process a different way.

19. What is the `category` dtype used for, and why does it help? It's used for columns with a small, repeating set of values (like department or country). It reduces memory usage significantly and can speed up group-by operations compared to storing the same values as `object` text.

20. What's the difference between data cleaning and data preprocessing? Data cleaning focuses narrowly on correctness (fixing missing values, duplicates, wrong types, typos). Data preprocessing is broader — it includes cleaning plus steps that prepare data specifically for modeling, like scaling, encoding, and feature engineering.

Intermediate (15)

21. What are MCAR, MAR, and MNAR? Why does the distinction matter? They describe *why* data is missing: MCAR (Missing Completely At Random) has no relationship to any data; MAR (Missing At Random) relates to other observed columns; MNAR (Missing Not At Random) relates to the missing value itself (e.g. high earners not disclosing income). It matters because MNAR missingness can make naive imputation actively misleading — the fact that it's missing is informative.

22. Why does mean imputation reduce variance, and when is that a problem? Every imputed value is exactly the average, so the imputed rows have zero spread around the mean, artificially shrinking the column's overall variance. This becomes a problem for any analysis or model that relies on variance being realistic, like correlation or variance-based feature selection.

23. Why is median often preferred over mean for skewed data like income or price? The mean is pulled toward extreme values, while the median represents the "typical" value regardless of how skewed the tail is, making it a more representative fill value or summary statistic for skewed distributions.

24. What's the practical difference between `where()` and boolean filtering (`df[condition]`)? Boolean filtering removes rows that don't match, changing the DataFrame's shape. `where()` keeps the same shape but replaces non-matching values with `NaN` (or a specified value) in place.

25. Why does `SettingWithCopyWarning` occur, and how do you avoid it? It occurs because a filtered/sliced DataFrame might be a view into the original rather than an independent copy, so editing it has ambiguous effects. Calling `.copy()` explicitly right after filtering guarantees an independent object and avoids the warning.

26. Why should `str.contains()` almost always be called with `na=False`? Because by default, `str.contains()` raises an error when it encounters a missing value in the column. Passing `na=False` tells it to simply treat missing values as "doesn't contain the pattern" instead of crashing.

27. When would you use `clip()` instead of removing outlier rows? When you want to limit the effect of extreme values on aggregates (like an average) without losing the rest of that row's information — `clip()` caps the value in place rather than deleting the row.

28. Explain the difference between `merge()` and `concat()` for combining data (as it relates to cleaning multi-source data). `merge()` joins two tables based on a shared key column, similar to a SQL JOIN, and is used when you're combining related entities. `concat()` simply stacks DataFrames together (usually adding rows), used when combining multiple files of the same structure, like monthly exports.

29. Why might `nunique()` overstate how many real categories exist in a column? Because inconsistent formatting — different casing, extra whitespace, or minor misspellings — creates multiple distinct string values that actually represent the same real-world category.

30. What's a practical difference between `map()` and `replace()` when cleaning categorical data? `map()` applies a full lookup and converts anything not in your dictionary to `NaN`, so it requires covering every value. `replace()` only changes the specific values you list and leaves everything else in the column untouched.

31. Why is `pd.Categorical()` with `ordered=True` useful for columns like "Low/Medium/High"? Without an explicit order, Pandas sorts such columns alphabetically (giving a nonsensical "High, Low, Medium" order). Defining the categories with `ordered=True` lets sorting and comparisons follow the real logical order instead.

32. How would you detect and handle time-zone inconsistencies when merging transaction data from two systems? Check whether each timestamp column is timezone-aware or naive, standardize all timestamps to a single timezone (typically UTC) using `tz_localize()`/`tz_convert()`, and confirm this before any date-based filtering or merging, since comparing aware and naive timestamps directly raises an error.

33. Why is `errors="coerce"` generally safer than `errors="raise"` during an exploratory cleaning pass? `coerce` lets the conversion continue past bad values (turning them into `NaN`/`NaT`) so you can see the full scope of the problem — how many rows are affected — instead of the whole script stopping at the very first bad value.

34. What's a good process for standardizing inconsistent country or category names? Inspect all unique values first, normalize case and whitespace to shrink accidental duplicates, then explicitly map the remaining true inconsistencies (typos, abbreviations) using a dictionary — inspect, normalize, then map, in that order.

35. Why can removing all duplicate rows sometimes delete real, valid data? Because two rows can be identical in every recorded column yet represent two genuinely separate real-world events — for example, a customer buying the same coffee twice in one day. The safer approach is deduplicating on a genuine business key (like a transaction ID) rather than the whole row.

Advanced (10)

36. How would you design a missing-value strategy for a dataset where missingness itself is informative (MNAR)? Rather than imputing blindly, consider adding a separate binary indicator column (`was_missing`) alongside the imputed value, so any model or analysis can still use the fact that it was missing as a signal, instead of that information being erased.

37. How would you decide, in a production pipeline, whether to drop, cap, or keep outliers, and how might that decision change between a linear regression model and a tree-based model? The decision should be driven by whether the outlier reflects a real event or an error, and by model sensitivity — linear models and distance-based methods (like k-means) are strongly affected by extreme values, so capping or removing may be necessary, while tree-based models (like random forests or gradient boosting) split on thresholds and are naturally robust to outliers, so they often don't require the same treatment.

38. How do you handle data cleaning differently for a one-time analysis versus a recurring production pipeline? A one-time analysis can involve manual inspection and ad-hoc fixes tailored to what you see. A production pipeline needs cleaning logic that's automated, reproducible, and defensive — validating assumptions (e.g. asserting no negative ages after cleaning) so it fails loudly rather than silently producing wrong output when new, unexpected dirty data arrives later.

39. What's the risk of applying the same IQR outlier thresholds across every group in a `groupby()`, versus computing them per group? A single global threshold ignores that different groups can have very different natural scales — e.g. luxury real estate versus starter homes — so a global IQR could flag an entire (legitimate) group as outliers. Computing IQR bounds per group (e.g., per city or per category) usually gives a more meaningful outlier definition.

40. How would you validate that a cleaning pipeline hasn't introduced bias into the dataset? Compare summary statistics (mean, distribution shape, category proportions) of the dataset before and after cleaning, and check whether missingness or outlier removal is disproportionately affecting certain subgroups (e.g. dropping rows tied to a particular region or demographic) rather than being evenly distributed.

41. Why might converting a column to the `category` dtype cause unexpected behavior in a data pipeline later on? If new data introduces a category not in the original fixed set, operations that append new values to a `category` column will either raise an error or convert them to `NaN`, since `category` dtype expects a closed set of allowed values unless you explicitly add new categories first.

42. How would you clean a dataset where the same real-world entity (e.g. a customer) has records with slightly different names, addresses, or IDs across systems (fuzzy duplicates)? Exact-match methods like `drop_duplicates()` won't catch these. This calls for fuzzy matching techniques (string similarity scores, phonetic matching, or record-linkage libraries) combined with business rules, and usually a manual review step for borderline matches rather than fully automated deduplication.

43. What's the tradeoff between cleaning data in Pandas versus doing it upstream in SQL or an ETL tool? Pandas cleaning is flexible and fast to iterate on for exploration, but doesn't scale well to very large datasets and isn't automatically shared across teams. Cleaning upstream (in SQL/ETL) ensures consistency for everyone who queries the table, but is slower to iterate on and requires more setup/permissions — many teams do a first pass upstream and finer-grained, analysis-specific cleaning in Pandas.

44. How would you handle a numeric column where "0" sometimes means a genuine zero and sometimes means missing/not-recorded? This requires domain investigation rather than a purely statistical answer — check whether "0" clusters suspiciously (e.g. every "0" also has other missing fields in that row) versus appearing naturally across otherwise-complete rows, and if ambiguous, treat suspicious zeros as missing (`mask()` them) and re-impute, documenting the assumption clearly for whoever uses the data next.

45. If you inherited a cleaning script with no documentation, what would you check before trusting its output? Whether it uses `inplace=True` in ways that could silently change data unexpectedly, whether it uses `errors="coerce"` without checking how many values were lost, whether it drops rows/columns without logging counts before and after, and whether it was written against assumptions (like a specific date format) that might not hold for newer data.

Chapter 18 — Complete Data Cleaning Roadmap (Most Important)

This is the step-by-step workflow to follow for **any** dataset, in **any** domain. Every earlier chapter is a deep dive into one node of this roadmap.

1. Load Dataset
↓
2. Understand Dataset (head, sample, describe)
↓
3. Check Shape (rows, columns)
↓
4. Inspect Data Types (dtypes, info())
↓
5. Inspect Missing Values (isnull().sum())
↓
6. Handle Missing Values (drop / impute)
↓
7. Check Duplicate Rows (duplicated())
↓
8. Remove/Investigate Duplicates
↓
9. Fix Data Types (astype, to_numeric, to_datetime)
↓
10. Clean Text Columns (.str methods)
↓
11. Handle Categories (replace, map, Categorical)
↓
12. Handle Outliers (IQR, Z-score, clip)
↓
13. Validate Numeric Ranges (age >= 0, price > 0, etc.)
↓
14. Clean Date Columns (to_datetime, dt accessor)
↓
15. Rename Columns (consistent, readable names)
↓
16. Final Quality Check (re-run shape/info/isnull/duplicated)
↓
17. Save Clean Dataset (new filename, never overwrite raw)

Step-by-Step Detail

1. Load Dataset

- *Why:* You can't clean what you haven't loaded correctly.
- *Functions:* `pd.read_csv()`, `pd.read_excel()`, `pd.read_json()`, `pd.read_parquet()`
- *Mistakes to avoid:* Not specifying `na_values` for custom missing markers; not checking encoding issues on non-English text.
- *When to skip:* Never — always the first step.

2-3. Understand Dataset & Check Shape

- *Why:* Sets your expectations — how big is this, what should it roughly look like?
- *Functions:* `head()`, `sample()`, `shape`, `describe()`
- *Mistakes to avoid:* Trusting `head()` alone on sorted data — use `sample()` too.
- *Industry practice:* Compare the row count against what you expected from the source system, if known (e.g. "we should have ~10,000 customers").

4. Inspect Data Types

- *Why:* Wrong types (numbers-as-text, dates-as-text) block almost every later step.
- *Functions:* `dtypes`, `info()`
- *Mistakes to avoid:* Assuming a column that *displays* as numbers in `head()` is actually numeric — always check `dtypes`.

5-6. Inspect & Handle Missing Values

- *Why:* Missing data breaks aggregations, visualizations, and models.
- *Functions:* `isnull()`, `notnull()`, `dropna()`, `fillna()`, `interpolate()`, `ffill()`, `bfill()`
- *Mistakes to avoid:* Blind `dropna()` with no `subset`; imputing without checking *why* it's missing first.
- *When to skip:* If a preliminary check shows zero missing values (rare, but do check rather than assume).

7-8. Check & Handle Duplicates

- *Why:* Duplicates silently inflate counts, sums, and averages.
- *Functions:* `duplicated()`, `drop_duplicates()`
- *Mistakes to avoid:* Deduplicating on the whole row instead of a real business key.
- *Industry practice:* Log how many duplicates were found and removed — don't delete silently.

9. Fix Data Types

- *Why:* Correct types enable correct math, correct sorting, correct filtering.
- *Functions:* `astype()`, `pd.to_numeric()`, `pd.to_datetime()`, `pd.to_timedelta()`, `convert_dtypes()`
- *Mistakes to avoid:* Converting before handling missing values (`NaN` can't become `int`).

10. Clean Text Columns

- *Why:* Inconsistent text breaks joins, filters, and category counts.
- *Functions:* `.str.strip()`, `.str.lower()`, `.str.replace()`, `.str.extract()`
- *Mistakes to avoid:* Fixing case but forgetting whitespace, or vice versa.

11. Handle Categories

- *Why:* Typo'd or inconsistent categories look like more diversity than actually exists.
- *Functions:* `replace()`, `map()`, `pd.Categorical()`
- *When to skip:* If the column is genuinely free text (like a review comment) rather than a fixed category set.

12-13. Handle Outliers & Validate Ranges

- *Why:* Impossible or extreme values distort statistics and can signal real data-entry errors.
- *Functions:* `quantile()`, `clip()`, boolean filtering, `mask()`
- *Mistakes to avoid:* Removing real rare events (fraud, luxury listings) that are the actual subject of the analysis.

14. Clean Date Columns

- *Why:* Broken dates break any time-based analysis (trends, cohorts, tenure).
- *Functions:* `pd.to_datetime()`, `.dt.year/month/day/weekday`
- *Mistakes to avoid:* Trusting the default date-format guess without checking the source convention.

15. Rename Columns

- *Why:* Consistent, readable names (`snake_case`, no spaces) make later code and collaboration easier.
- *Functions:* `rename(columns={...})`

- *Industry practice:* Standardize naming conventions across all datasets in a project.

16. Final Quality Check

- *Why:* Confirms the dataset is actually clean before anyone relies on it.
- *Functions:* Re-run `shape()`, `info()`, `isnull().sum()`, `duplicated().sum()`
- *Mistakes to avoid:* Skipping this because "I already cleaned it" — always verify, don't assume.

17. Save Clean Dataset

- *Why:* Preserves your work and gives downstream steps (EDA, modeling) a stable, trusted input.
- *Functions:* `to_csv(index=False)`, `to_excel()`, `to_parquet()`
- *Mistakes to avoid:* Overwriting the raw source file — always save under a new name.

🎯 **Interview Tip:** Memorizing this 17-step roadmap (even the shorter 8-step version from the Table of Contents) gives you a ready, structured answer to almost any "how would you approach a new messy dataset?" question.

Chapter 19 — Practice Datasets (Solve Independently)

Each scenario below describes a small, intentionally messy dataset. Try to write the cleaning code yourself before checking your approach against the earlier chapters. No solutions are given here on purpose — that's the point.

Practice 1 — Housing Prices

```
python

housing = pd.DataFrame({
    "listing_id": [1, 2, 3, 3, 4],
    "price": ["$250,000", "310000", "$1,200,000", "$1,200,000", "N/A"],
    "bedrooms": [3, 4, -1, -1, 2],
    "listed_date": ["01/03/2023", "2023-05-11", "15-06-2023", "15-06-2023", "invalid"]
})
```

Problems to find: currency symbols, a placeholder `"N/A"`, an impossible negative bedroom count, duplicate listing, three different date formats.

Practice 2 — Customer Information

```
python

customers = pd.DataFrame({
    "email": [" John@gmail.com", "MARY@YAHOO.COM", "invalid-email", None, "sam@corp"],
    "phone": ["9876543210", "98765-43210", "987654321", "9876543210x", "9999999999"],
    "country": ["USA", "U.S.A.", "United States", "IN", "India"]
})
```

Problems to find: inconsistent casing/whitespace in emails, an invalid email, missing email, inconsistent phone formatting, an incomplete phone number, inconsistent country naming.

Practice 3 — Sales Data

```
python
```

```
sales = pd.DataFrame({
    "order_id": [501, 502, 503, 504],
    "quantity": [5, -2, "ten", 3],
    "discount": ["10%", "0.15", "20", None],
    "sale_date": ["2024-01-15", "01/16/2024", "17-01-2024", "2024/01/18"]
})
```

Problems to find: negative quantity, a text value where a number is expected, inconsistent discount format (percent string vs. decimal vs. ambiguous whole number), four different date formats.

Practice 4 — Medical Records

```
python

medical = pd.DataFrame({
    "patient_id": [1, 2, 3, 4, 5],
    "age": [45, -3, 250, 30, None],
    "blood_group": ["A+", "a+", "O NEGATIVE", "AB+", "B+ "],
    "height_cm": ["165cm", "180", "5.9ft", "172cm", None]
})
```

Problems to find: negative and impossible ages, a missing age, inconsistent blood group formatting, mixed units in height (cm vs. feet).

Practice 5 — Employee Records

```
python

employees = pd.DataFrame({
    "emp_id": [1, 2, 2, 3, 4],
    "department": ["Sales", "sales", "sales", "Engineering", "HR "],
    "salary": [50000, 99999999, 62000, 58000, None],
    "join_date": ["2020-01-10", "2020-01-10", "2020-01-10", "2021-99-01", "2019-05-"]
})
```

Problems to find: duplicate employee row, inconsistent department casing, a misspelled department, an implausibly large salary (likely a data-entry error), a missing salary, an invalid date (month 99).

Practice 6 — Restaurant Reviews

```
python

reviews = pd.DataFrame({
    "review_id": [1, 2, 3, 4],
    "rating": ["5 stars", "4.0", "Excellent", "2"],
    "review_text": [" Great food!!", "OK experience.", None, "Terrible... never ag"],
    "visit_date": ["2023-11-05", "2023/11/06", "06-11-2023", "invalid"]
})
```

Problems to find: inconsistent rating formats (text vs. number vs. word), messy whitespace/punctuation in review text, missing review text, mixed date formats and one invalid date.

Practice 7 — Banking Transactions

```
python

transactions = pd.DataFrame({
    "txn_id": ["T1", "T2", "T2", "T3", "T4"],
    "amount": ["₹5,000", "-2000", "₹15,00,000", "500", "₹0"],
    "type": ["debit", "Credit", "DEBIT", "credit ", "Debit"],
    "timestamp": ["2024-02-01 10:00", "2024-02-01 10:00", "2024-02-01 10:00", "2024-02-01 10:00", "2024-02-01 10:00"]
})
```

Problems to find: duplicate transaction ID, currency symbol and Indian-style comma grouping, a negative amount (is that valid for a debit, or an error?), inconsistent casing/whitespace in `type`, a missing timestamp.

Practice 8 — Product Inventory

```
python

inventory = pd.DataFrame({
    "sku": ["ELEC-001", "elec-002", "FOOD-010", "FOOD-010", "CLOTH-05"],
    "stock_qty": ["150", "N/A", "-10", "-10", "0"],
    "unit_weight": ["2.5kg", "500g", "1.2 kg", "1.2 kg", None],
    "stock_level": ["High", "high", "LOW", "LOW", "Medium"]
})
```

Problems to find: inconsistent SKU casing, a placeholder `"N/A"` in a numeric field, a negative stock quantity, duplicate SKU rows, mixed weight units (kg vs. g), inconsistent category casing, a missing weight.

Practice 9 — E-commerce Orders

```
python

orders = pd.DataFrame({
    "order_id": [9001, 9002, 9003, 9004],
    "order_total": ["$120.50", "45", "$-10.00", "$99999999"],
    "customer_email": ["a@b.com", "BAD EMAIL", "c@d.com", None],
    "status": ["Delivered", "delivered", "CANCELLED", "shipped "]
})
```

Problems to find: currency symbol, a negative order total, an implausibly large total (likely an error), an invalid email format, a missing email, inconsistent status casing/whitespace.

💡 **Pro Tip:** For each practice dataset, try writing out your reasoning in words first ("I would drop this row because...") before writing code — in interviews, your reasoning is usually judged as closely as your syntax.

Chapter 20 — Final Revision Sheet (10-15 Minute Lightning Review)

The One-Paragraph Summary

Data cleaning turns raw, inconsistent, incomplete data into trustworthy, analysis-ready data. The standard order is: **load** → **understand** → **check duplicates & types** → **handle missing values** → **fix types** → **clean text/categories** → **handle outliers** → **validate ranges** → **clean dates** → **rename** → **final check** → **save**. Every decision (drop vs. impute, remove vs. cap) should be driven by *why* the problem exists and what the data will be used for — not by memorized syntax alone.

Missing Value Strategy (30-Second Version)

Column mostly missing (>50-60%)? -> Drop the COLUMN
Row missing a critical ID, rows few? -> Drop the ROW
Numeric, normal-ish? -> Mean
Numeric, skewed/has outliers? -> Median
Categorical? -> Mode
Missing has real meaning? -> Constant value ("Unknown", 0)
Time-ordered, slow-changing? -> ffill() / bfill()
Time-ordered, smooth trend? -> interpolate()

Outlier Strategy (30-Second Version)

Skewed data -> IQR method ($Q1 - 1.5 \cdot IQR$, $Q3 + 1.5 \cdot IQR$)
Roughly normal data -> Z-score ($|z| > 3$)
Business-rule based -> Percentile trim (e.g. top/bottom 1%)
Want to keep the row but limit the value -> clip()
Outlier IS the signal (fraud, rare events) -> KEEP it

Most Important Functions (Memorize These)

```
python  
  
df.info(); df.describe(); df.isnull().sum(); df.duplicated().sum()  
df.dropna(subset=[...]); df.fillna(value)  
df.drop_duplicates(subset=[...], keep='first')  
pd.to_numeric(series, errors='coerce')  
pd.to_datetime(series, format=..., errors='coerce')  
series.str.strip().str.lower()  
df.loc[condition, 'column']  
df[(cond1) & (cond2)]  
df["col"].clip(lower=, upper=)  
df.to_csv("clean.csv", index=False)
```

Common Syntax Gotchas

Wrong	Right	Why
<code>df["a","b"]</code>	<code>df[["a","b"]]</code>	Multiple columns need a list
<code>df[cond1 and cond2]</code>	<code>df[(cond1) & (cond2)]</code>	Element-wise needs <code>&</code> / <code> </code> / <code>~</code> with parentheses
<code>df.dropna(inplace=True)</code> then reassigning	Just <code>df = df.dropna()</code>	<code>inplace=True</code> returns <code>None</code>
<code>df.astype(bool)</code> on "False" text	<code>.map({"True": True, "False": False})</code>	Any non-empty string is truthy
<code>filtered["x"] = y</code> after slicing	<code>filtered = df[cond].copy(); filtered["x"] = y</code>	Avoids <code>SettingWithCopyWarning</code>

The Roadmap, One More Time

Load → Understand → Shape → Dtypes → Missing Values → Duplicates →
Fix Types → Clean Text → Handle Categories → Handle Outliers →
Validate Ranges → Clean Dates → Rename Columns → Final Check → Save

Interview Tips Recap

- Always explain **why**, not just **how** — judgment matters more than syntax recall.
- Know the MCAR/MAR/MNAR distinction; it signals real statistical understanding.
- State the "1.5 × IQR" outlier rule from memory.
- Mention `errors="coerce"` when discussing type conversion — it shows awareness of real-world messy data.
- Never claim "always drop duplicates" or "always fill with the mean" — show that the right answer depends on context.

Best Practices Recap

- Never overwrite raw data — always save cleaned output under a new name.
- Investigate duplicates before deleting them.
- Check `.unique()` before writing any regex or mapping dictionary.
- Re-run your quality checks (`shape`, `info()`, `isnull().sum()`, `duplicated().sum()`) after every major step.
- Document *why* you made each cleaning decision — future-you (or a teammate) will need to know.

Mistakes to Avoid Recap

- Blind `dropna()` with no `subset`
- Converting types before handling missing values
- Chained indexing instead of `.loc[]`
- Trusting `pd.to_datetime()`'s default format guess
- Treating every duplicate row as automatically wrong

You've now covered the full journey: from raw, messy exports across nine different real-world domains, to a clean, validated, analysis-ready dataset — and the reasoning behind every step along the way. Good luck with your interviews and your projects.